Week-8 (DAY-2) – Handson – Keerthivasan R

📌 Problem Statement

In modern web applications, securing user data and restricting unauthorized access is essential. Traditional cookie-based authentication works well for simple applications but becomes difficult to manage in applications that consume Web APIs or follow a layered architecture.

Many organizations face the following issues:

- Unauthorized users accessing restricted pages
- Lack of proper session management
- No role-based page restrictions (Admin/User)
- Difficulty integrating MVC frontend with secure Web APIs
- Security vulnerabilities due to improper authentication handling

To overcome these challenges, a **secure, token-based authentication system using JWT in ASP.NET Core MVC** must be implemented.

🎯 Project Objective

Develop a secure ASP.NET Core MVC application that:

- Allows users to Register and Login
- Generates a JWT token after successful login
- Stores the token securely (HttpOnly cookie or session)
- Uses the token to access protected controllers/actions
- Implements Role-Based Authorization (Admin & User)
- Restricts access to specific views based on roles
- Connects with a database using Entity Framework Core

🛠️ Technologies to be Used

- ASP.NET Core MVC
- ASP.NET Core Web API (for token generation)
- JWT (JSON Web Token)
- Entity Framework Core
- SQL Server
- Role-Based Authorization

📌 Functional Requirements

1. User Registration Page
2. User Login Page

3. JWT Token Generation
4. Secure Dashboard Page (Authorized users only)
5. Admin Panel (Admin role only)
6. Logout Functionality
7. Proper error handling (401 Unauthorized, 403 Forbidden)

📌 Non-Functional Requirements

- Secure password storage (hashed passwords)
- Token expiration handling
- Scalable and stateless authentication
- Clean MVC architecture

📌 Expected Outcome

At the end of the project:

- Users can securely log in
- JWT token will control access to MVC pages
- Only authorized users can access protected views
- Admin-only pages will be restricted properly
- Application follows secure authentication standards
- MVC integration diagram

CODE :

📁 Models/ApplicationUser.cs

```
using Microsoft.AspNetCore.Identity;

namespace SecureJWTMVC.Models
{
    public class ApplicationUser : IdentityUser
    {
        public string? FullName { get; set; }
        public DateTime CreatedAt { get; set; } = DateTime.Now;
    }
}
```

📁 Data/ApplicationDbContext.cs

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using SecureJWTMVC.Models;

namespace SecureJWTMVC.Data
{
    // Primary constructor added to the class declaration
    public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : IdentityDbContext<ApplicationUser>(options)
    {
        protected override void OnModelCreating(ModelBuilder builder)
        {
```

```
            base.OnModelCreating(builder);
        }
    }
}
```

📁 Models/JwtSettings.cs

```
namespace SecureJWTMVC.Models
{
    public class JwtSettings
    {
        public string SecretKey { get; set; } = string.Empty;
        public string Issuer { get; set; } = string.Empty;
        public string Audience { get; set; } = string.Empty;
        public int ExpirationMinutes { get; set; }
    }
}
```

📁 appsettings.json

```
{
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    },
    "AllowedHosts": "*",
    "ConnectionStrings": {
        "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=SecureJWTMVC_DB;Trusted_Connection=True;MultipleActiv
eResultSets=true"
    },
    "JwtSettings": {
        "SecretKey": "YourSuperSecretKeyHere123!@#ThatIsAtLeast32CharsLong",
        "Issuer": "SecureJWTMVC",
        "Audience": "SecureJWTMVCUsers",
        "ExpirationMinutes": 60
    }
}
```

📁 Data/SeedData.cs

```
using Microsoft.AspNetCore.Identity;
using SecureJWTMVC.Models;

namespace SecureJWTMVC.Data
{
    public static class SeedData
    {
        public static async Task InitializeAsync(IServiceProvider serviceProvider)
        {
            var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
            var userManager =
serviceProvider.GetRequiredService<UserManager<ApplicationUser>>();

            // Create roles
            string[] roleNames = [ "Admin", "User" ];
            foreach (var roleName in roleNames)
            {
                if (!await roleManager.RoleExistsAsync(roleName))
                {
                    await roleManager.CreateAsync(new IdentityRole(roleName));
                }
            }

            // Create admin user
```

```
            var adminEmail = "admin@secureapp.com";
            var adminUser = await userManager.FindByEmailAsync(adminEmail);

            if (adminUser == null)
            {
                adminUser = new ApplicationUser
                {
                    UserName = adminEmail,
                    Email = adminEmail,
                    FullName = "System Administrator",
                    EmailConfirmed = true
                };

                var result = await userManager.CreateAsync(adminUser, "Admin@123");

                if (result.Succeeded)
                {
                    await userManager.AddToRoleAsync(adminUser, "Admin");
                }
            }
        }
    }
}
```

📁 Controllers/AccountController.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using SecureJWTMVC.Models;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace SecureJWTMVC.Controllers
{
    public class AccountController : Controller
    {
        private readonly UserManager<ApplicationUser> _userManager;
        private readonly SignInManager<ApplicationUser> _signInManager;
        private readonly RoleManager<IdentityRole> _roleManager;
        private readonly JwtSettings _jwtSettings;

        public AccountController(
            UserManager<ApplicationUser> userManager,
            SignInManager<ApplicationUser> signInManager,
            RoleManager<IdentityRole> roleManager,
            IOptions<JwtSettings> jwtSettings)
        {
            _userManager = userManager;
            _signInManager = signInManager;
            _roleManager = roleManager;
            _jwtSettings = jwtSettings.Value;
        }

        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Register(RegisterViewModel model)
        {
            if (ModelState.IsValid)
            {
                var user = new ApplicationUser
```

```
                {
                    UserName = model.Email,
                    Email = model.Email,
                    FullName = model.FullName
                };

                var result = await _userManager.CreateAsync(user, model.Password);

                if (result.Succeeded)
                {
                    // Assign default "User" role
                    await _userManager.AddToRoleAsync(user, "User");

                    TempData["Success"] = "Registration successful! Please login.";
                    return RedirectToAction(nameof(Login));
                }

                foreach (var error in result.Errors)
                {
                    ModelState.AddModelError(string.Empty, error.Description);
                }
            }
            return View(model);
        }

        [HttpGet]
        public IActionResult Login()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Login(LoginViewModel model)
        {
            if (ModelState.IsValid)
            {
                var user = await _userManager.FindByEmailAsync(model.Email);

                if (user != null && await _userManager.CheckPasswordAsync(user,
model.Password))
                {
                    // Generate JWT Token
                    var token = await GenerateJwtToken(user);

                    // Store token in HttpOnly Cookie
                    Response.Cookies.Append("AuthToken", token, new CookieOptions
                    {
                        HttpOnly = true,
                        Secure = true,
                        SameSite = SameSiteMode.Strict,
                        Expires = DateTime.UtcNow.AddMinutes(_jwtSettings.ExpirationMinutes)
                    });

                    // Store user info in session (optional)
                    HttpContext.Session.SetString("UserEmail", user.Email!);
                    HttpContext.Session.SetString("UserFullName", user.FullName!);

                    return RedirectToAction("Index", "Dashboard");
                }

                ModelState.AddModelError(string.Empty, "Invalid login attempt.");
            }
            return View(model);
        }

        private async Task<string> GenerateJwtToken(ApplicationUser user)
        {
            var userRoles = await _userManager.GetRolesAsync(user);
```

```
            var claims = new List<Claim>
            {
                new(ClaimTypes.NameIdentifier, user.Id),
                new(ClaimTypes.Email, user.Email!),
                new(ClaimTypes.Name, user.FullName ?? user.Email!),
                new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
            };

            // Add roles to claims
            foreach (var role in userRoles)
            {
                claims.Add(new Claim(ClaimTypes.Role, role));
            }

            var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtSettings.SecretKey));
            var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

            var token = new JwtSecurityToken(
                issuer: _jwtSettings.Issuer,
                audience: _jwtSettings.Audience,
                claims: claims,
                expires: DateTime.UtcNow.AddMinutes(_jwtSettings.ExpirationMinutes),
                signingCredentials: credentials
            );

            return new JwtSecurityTokenHandler().WriteToken(token);
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Logout()
        {
            await _signInManager.SignOutAsync();
            Response.Cookies.Delete("AuthToken");
            HttpContext.Session.Clear();
            return RedirectToAction("Index", "Home");
        }

        [HttpGet]
        public IActionResult AccessDenied()
        {
            return View();
        }
    }
}
```

📁 Models/RegisterViewModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace SecureJWTMVC.Models
{
    public class RegisterViewModel
    {
        [Required]
        [EmailAddress]
        [Display(Name = "Email")]
        public string Email { get; set; } = string.Empty;

        [Required]
        [Display(Name = "Full Name")]
        public string FullName { get; set; } = string.Empty;

        [Required]
        [StringLength(100, ErrorMessage = "The {0} must be at least {2} characters long.",
MinimumLength = 6)]
        [DataType(DataType.Password)]
        [Display(Name = "Password")]
```

```
        public string Password { get; set; } = string.Empty;

        [DataType(DataType.Password)]
        [Display(Name = "Confirm password")]
        [Compare("Password", ErrorMessage = "The password and confirmation password do not
match.")]
        public string ConfirmPassword { get; set; } = string.Empty;
    }
}
```

📁 Models/LoginViewModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace SecureJWTMVC.Models
{
    public class LoginViewModel
    {
        [Required]
        [EmailAddress]
        [Display(Name = "Email")]
        public string Email { get; set; } = string.Empty;

        [Required]
        [DataType(DataType.Password)]
        [Display(Name = "Password")]
        public string Password { get; set; } = string.Empty;

        [Display(Name = "Remember me?")]
        public bool RememberMe { get; set; }
    }
}
```

📁 Views/Shared/_Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - Secure JWT App</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-dark bg-dark border-
bottom box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-
action="Index">Secure JWT App</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-
bs-target=".navbar-collapse">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-
between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-light" asp-controller="Home" asp-
action="Index">Home</a>
                        </li>

                        @* Use ?. and == true to safely check authentication *@
                        @if (User.Identity?.IsAuthenticated == true)
                        {
                            <li class="nav-item">
                                <a class="nav-link text-light" asp-controller="Dashboard" asp-
action="Index">Dashboard</a>
```

```
                            </li>

                            @if (User.IsInRole("Admin"))
                            {
                                <li class="nav-item">
                                    <a class="nav-link text-warning" asp-controller="Admin"
asp-action="Index">Admin Panel</a>
                                </li>
                            }
                        }
                    </ul>

                    <ul class="navbar-nav">
                        @* Line 37 fix: Added ?. and == true *@
                        @if (User.Identity?.IsAuthenticated == true)
                        {
                            <li class="nav-item">
                                @* Safely access Name with ? *@
                                <span class="nav-link text-info">Welcome,
@(User.Identity?.Name ?? "User")!</span>
                            </li>
                            <li class="nav-item">
                                <form asp-controller="Account" asp-action="Logout"
method="post" class="form-inline">
                                    <button type="submit" class="nav-link btn btn-link text-
light">Logout</button>
                                </form>
                            </li>
                        }
                        else
                        {
                            <li class="nav-item">
                                <a class="nav-link text-light" asp-controller="Account" asp-
action="Register">Register</a>
                            </li>
                            <li class="nav-item">
                                <a class="nav-link text-light" asp-controller="Account" asp-
action="Login">Login</a>
                            </li>
                        }
                    </ul>
                </div>
            </div>
        </nav>
    </header>

    <div class="container">
        <main role="main" class="pb-3">
            @if (TempData["Success"] != null)
            {
                <div class="alert alert-success alert-dismissible fade show" role="alert">
                    @TempData["Success"]
                    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
                </div>
            }

            @if (TempData["Error"] != null)
            {
                <div class="alert alert-danger alert-dismissible fade show" role="alert">
                    @TempData["Error"]
                    <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
                </div>
            }

            @RenderBody()
        </main>
    </div>

    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
```

```
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

📁 Views/Account/Login.cshtml

```
@model LoginViewModel

@{
    ViewData["Title"] = "Login";
}

<div class="row justify-content-center">
    <div class="col-md-6">
        <div class="card">
            <div class="card-header bg-primary text-white">
                <h3 class="text-center">Login</h3>
            </div>
            <div class="card-body">
                <form asp-action="Login" method="post">
                    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

                    <div class="form-group mb-3">
                        <label asp-for="Email" class="form-label"></label>
                        <input asp-for="Email" class="form-control" />
                        <span asp-validation-for="Email" class="text-danger"></span>
                    </div>

                    <div class="form-group mb-3">
                        <label asp-for="Password" class="form-label"></label>
                        <input asp-for="Password" class="form-control" />
                        <span asp-validation-for="Password" class="text-danger"></span>
                    </div>

                    <div class="form-group mb-3">
                        <div class="form-check">
                            <input asp-for="RememberMe" class="form-check-input" />
                            <label asp-for="RememberMe" class="form-check-label"></label>
                        </div>
                    </div>

                    <div class="form-group">
                        <button type="submit" class="btn btn-primary w-100">Login</button>
                    </div>
                </form>

                <div class="mt-3 text-center">
                    <p>Don't have an account? <a asp-action="Register">Register here</a></p>
                    <hr />
                    <small class="text-muted">Demo Admin: admin@secureapp.com /
Admin@123</small>
                </div>
            </div>
        </div>
    </div>
</div>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}
```

Views/Account/Register.cshtml

```
@model RegisterViewModel

@{
    ViewData["Title"] = "Register";
}

<div class="row justify-content-center">
    <div class="col-md-6">
        <div class="card">
            <div class="card-header bg-success text-white">
                <h3 class="text-center">Register New Account</h3>
            </div>
            <div class="card-body">
                <form asp-action="Register" method="post">
                    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

                    <div class="form-group mb-3">
                        <label asp-for="FullName" class="form-label"></label>
                        <input asp-for="FullName" class="form-control" />
                        <span asp-validation-for="FullName" class="text-danger"></span>
                    </div>

                    <div class="form-group mb-3">
                        <label asp-for="Email" class="form-label"></label>
                        <input asp-for="Email" class="form-control" />
                        <span asp-validation-for="Email" class="text-danger"></span>
                    </div>

                    <div class="form-group mb-3">
                        <label asp-for="Password" class="form-label"></label>
                        <input asp-for="Password" class="form-control" />
                        <span asp-validation-for="Password" class="text-danger"></span>
                    </div>

                    <div class="form-group mb-3">
                        <label asp-for="ConfirmPassword" class="form-label"></label>
                        <input asp-for="ConfirmPassword" class="form-control" />
                        <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
                    </div>

                    <div class="form-group">
                        <button type="submit" class="btn btn-success w-100">Register</button>
                    </div>
                </form>

                <div class="mt-3 text-center">
                    <p>Already have an account? <a asp-action="Login">Login here</a></p>
                </div>
            </div>
        </div>
    </div>
</div>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}
```

📁 Views/Account/AccessDenied.cshtml

```
@{
    ViewData["Title"] = "Access Denied";
}

<div class="text-center">
    <div class="alert alert-danger">
        <h1 class="display-4">Access Denied</h1>
        <p>You don't have permission to access this page.</p>
        <a asp-controller="Home" asp-action="Index" class="btn btn-primary">Return to Home</a>
    </div>
</div>
```

📁 Controllers/DashboardController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace SecureJWTMVC.Controllers
{
    [Authorize]
    public class DashboardController : Controller
    {
        public IActionResult Index()
        {
            ViewBag.UserEmail = User.Identity?.Name;
            ViewBag.UserRoles = User.Claims
                .Where(c => c.Type == System.Security.Claims.ClaimTypes.Role)
                .Select(c => c.Value)
                .ToList();

            return View();
        }
    }
}
```

📁 Views/Dashboard/Index.cshtml

```
@{
    ViewData["Title"] = "Dashboard";
}

<div class="container">
    <div class="row">
        <div class="col-md-12">
            <div class="card">
                <div class="card-header bg-info text-white">
                    <h3>Welcome to Your Dashboard</h3>
                </div>
                <div class="card-body">
                    <h4>User Information</h4>
                    <p><strong>Email:</strong> @ViewBag.UserEmail</p>
                    <p><strong>Roles:</strong> @string.Join(", ", ViewBag.UserRoles)</p>

                    <hr />

                    <div class="alert alert-success">
                        <h5>☑ JWT Token Active</h5>
                        <p>You are authenticated using JWT token stored in HttpOnly
cookie.</p>
                        <p>Token expires in 60 minutes.</p>
                    </div>

                    <div class="row mt-4">
                        <div class="col-md-6">
                            <div class="card">
                                <div class="card-body">
```

```
                                <h5>Protected Content</h5>
                                <p>This page is only accessible to authenticated
users.</p>
                            </div>
                        </div>
                    </div>

                    @if (User.IsInRole("Admin"))
                    {
                        <div class="col-md-6">
                            <div class="card bg-warning">
                                <div class="card-body">
                                    <h5>Admin Access</h5>
                                    <p>You have admin privileges!</p>
                                    <a asp-controller="Admin" asp-action="Index"
class="btn btn-dark">Go to Admin Panel</a>
                                </div>
                            </div>
                        </div>
                    }
                </div>
            </div>
        </div>
    </div>
</div>
```

📁 Controllers/AdminController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using SecureJWTMVC.Models;

namespace SecureJWTMVC.Controllers
{
    [Authorize(Roles = "Admin")]
    // Primary constructor defined
    public class AdminController(UserManager<ApplicationUser> userManager,
RoleManager<IdentityRole> roleManager)
        : Controller
    {
        public IActionResult Index()
        {
            // Parameters are accessed directly
            ViewBag.TotalUsers = userManager.Users.Count();
            ViewBag.TotalRoles = roleManager.Roles.Count();
            return View();
        }

        public IActionResult Users()
        {
            var users = userManager.Users.ToList();
            return View(users);
        }
    }
}
```

📁 Views/Admin/Index.cshtml

```
@{
    ViewData["Title"] = "Admin Panel";
}

<div class="container">
    <div class="alert alert-warning">
        <h3>🔐 Admin Panel - Restricted Access</h3>
        <p>Only users with <strong>Admin</strong> role can access this page.</p>
    </div>
```

```
    <div class="row">
        <div class="col-md-4">
            <div class="card bg-primary text-white">
                <div class="card-body">
                    <h3>@ViewBag.TotalUsers</h3>
                    <p>Total Users</p>
                </div>
            </div>
        </div>

        <div class="col-md-4">
            <div class="card bg-success text-white">
                <div class="card-body">
                    <h3>@ViewBag.TotalRoles</h3>
                    <p>Total Roles</p>
                </div>
            </div>
        </div>

        <div class="col-md-4">
            <div class="card bg-info text-white">
                <div class="card-body">
                    <h3>JWT</h3>
                    <p>Token Active</p>
                </div>
            </div>
        </div>
    </div>

    <div class="mt-4">
        <div class="card">
            <div class="card-header">
                <h4>Admin Actions</h4>
            </div>
            <div class="card-body">
                <a asp-action="Users" class="btn btn-primary">Manage Users</a>
            </div>
        </div>
    </div>
</div>
```

📁 Views/Admin/Users.cshtml

```
@model IEnumerable<SecureJWTMVC.Models.ApplicationUser>

@{
    ViewData["Title"] = "User Management";
}

<div class="container">
    <div class="card">
        <div class="card-header bg-warning">
            <h3>User Management</h3>
        </div>
        <div class="card-body">
            <table class="table table-striped">
                <thead>
                    <tr>
                        <th>Email</th>
                        <th>Full Name</th>
                        <th>Created At</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach (var user in Model)
                    {
                        <tr>
                            <td>@user.Email</td>
```

```
                        <td>@user.FullName</td>
                        <td>@user.CreatedAt.ToString("yyyy-MM-dd HH:mm")</td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
</div>
</div>
```

📁 Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace SecureJWTMVC.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        public IActionResult Privacy()
        {
            return View();
        }
    }
}
```

📁 Views/Home/Index.cshtml

```
@{
    ViewData["Title"] = "Home Page";
}

<div class="text-center">
    <h1 class="display-4">Welcome to Secure JWT Authentication</h1>
    <p>Learn about <a href="https://jwt.io/">JWT Tokens</a>.</p>
</div>

<div class="row mt-5">
    <div class="col-md-6">
        <div class="card">
            <div class="card-body">
                <h3>🔐 Secure Authentication</h3>
                <p>This application uses JWT (JSON Web Token) for secure authentication.</p>
                <ul>
                    <li>Tokens stored in HttpOnly cookies</li>
                    <li>Role-based authorization</li>
                    <li>Protected routes</li>
                    <li>Stateless authentication</li>
                </ul>
                @if (User.Identity?.IsAuthenticated != true)
                {
                    <a asp-controller="Account" asp-action="Login" class="btn btn-
primary">Login to Access Dashboard</a>
                }
            </div>
        </div>
    </div>

    <div class="col-md-6">
        <div class="card">
            <div class="card-body">
                <h3>👥 Demo Accounts</h3>
                <p><strong>Admin:</strong> admin@secureapp.com / Admin@123</p>
                <p><strong>User:</strong> Register a new account</p>
                <hr />
```

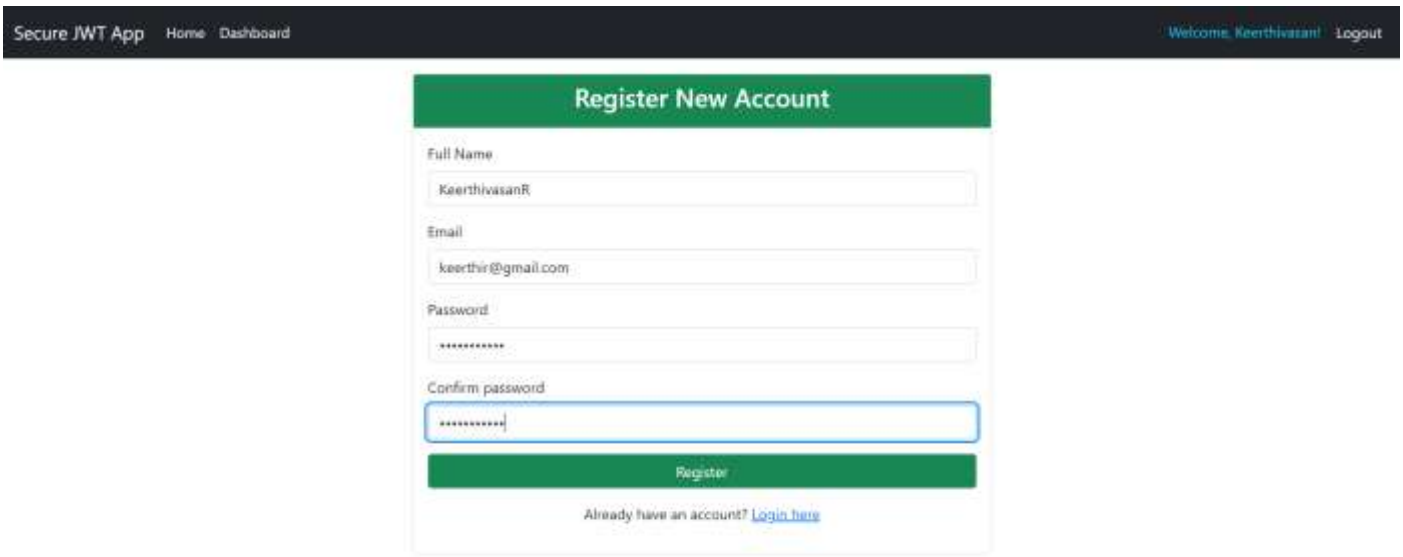
```
                <a asp-controller="Account" asp-action="Register" class="btn btn-
success">Create New Account</a>
            </div>
        </div>
    </div>
</div>
```

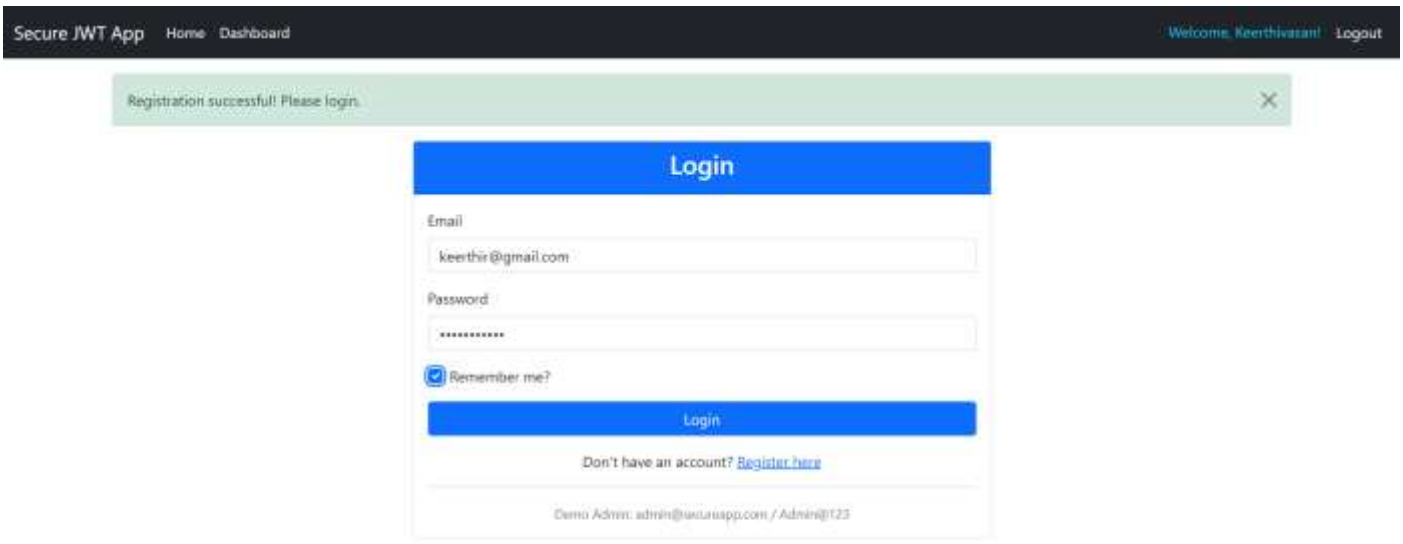
Program.cs

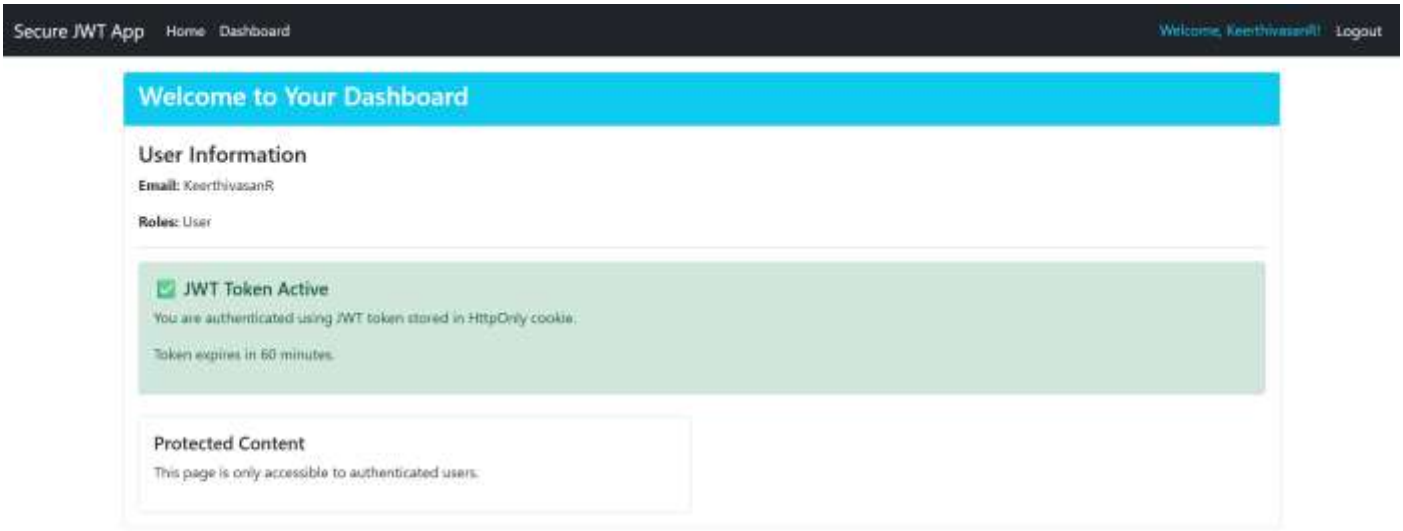
```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using SecureJWTMVC.Data;
using SecureJWTMVC.Models;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

// Configure Database Context
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Configure Identity
builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

// Configure JWT Settings
var jwtSettings = builder.Configuration.GetSection("JwtSettings");
var secretKey = Encoding.ASCII.GetBytes(jwtSettings["SecretKey"]!);

builder.Services.Configure<JwtSettings>(jwtSettings);

// Configure Authentication with JWT
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.SaveToken = true;
    options.RequireHttpsMetadata = false;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(secretKey),
        ClockSkew = TimeSpan.Zero
    };

    // For MVC - extract token from cookie
```

```
    options.Events = new JwtBearerEvents
    {
        OnMessageReceived = context =>
        {
            context.Token = context.Request.Cookies["AuthToken"];
            return Task.CompletedTask;
        }
    };
});
// Use AddAuthorizationBuilder to register authorization services and construct policies
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("AdminOnly", policy => policy.RequireRole("Admin"))
    .AddPolicy("UserOnly", policy => policy.RequireRole("User", "Admin"));
var app = builder.Build();
// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseSession();
app.UseAuthentication(); // Important: Add this before Authorization
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
// Seed Roles and Admin User
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    await SeedData.InitializeAsync(services);
}
app.Run();
```

OUTPUT:


Secure JWT App
Home
Dashboard
Welcome, Keerthivasan!
Logout
Register New Account
Full Name
KeerthivasanR
Email
keerthir@gmail.com
Password
Confirm password
Register
Already have an account? Login here


Secure JWT App
Home
Dashboard
Welcome, Keerthivasan!
Logout
Registration successful! Please login.
Login
Email
keerthir@gmail.com
Password
Remember me?
Login
Don't have an account? Register here



	Id	FullName	CreatedAt	UserName	NormalizedUs...	Email	NormalizedEm...	EmailConfirmed	PasswordHash	SecurityStamp	ConcurrencySt...
	37d6d1de-d4f6...	Keerthivasan	11-04-2026 13:...	keerthi@gmail....	KEERTHI@GMA...	keerthi@gmail....	KEERTHI@GMA...	False	AQAAAAIAAYa...	TVO6SKCDXPKP...	c1907f32-9eea-...
	63222773-892b...	KeerthivasanR	11-04-2026 13:...	keerthir@gmail....	KEERTHIR@GM...	keerthir@gmail....	KEERTHIR@GM...	False	AQAAAAIAAYa...	577OR6OLCQJZ...	5467cf34-115e-...
	90edc34a-5f83-...	System Adminis...	11-04-2026 13:...	admin@securea...	ADMIN@SECU...	admin@securea...	ADMIN@SECU...	True	AQAAAAIAAYa...	3CMEXZ3CCKR...	d0cabba4-3339...